第七周作业 -1

第七周作业 -1

基础作业

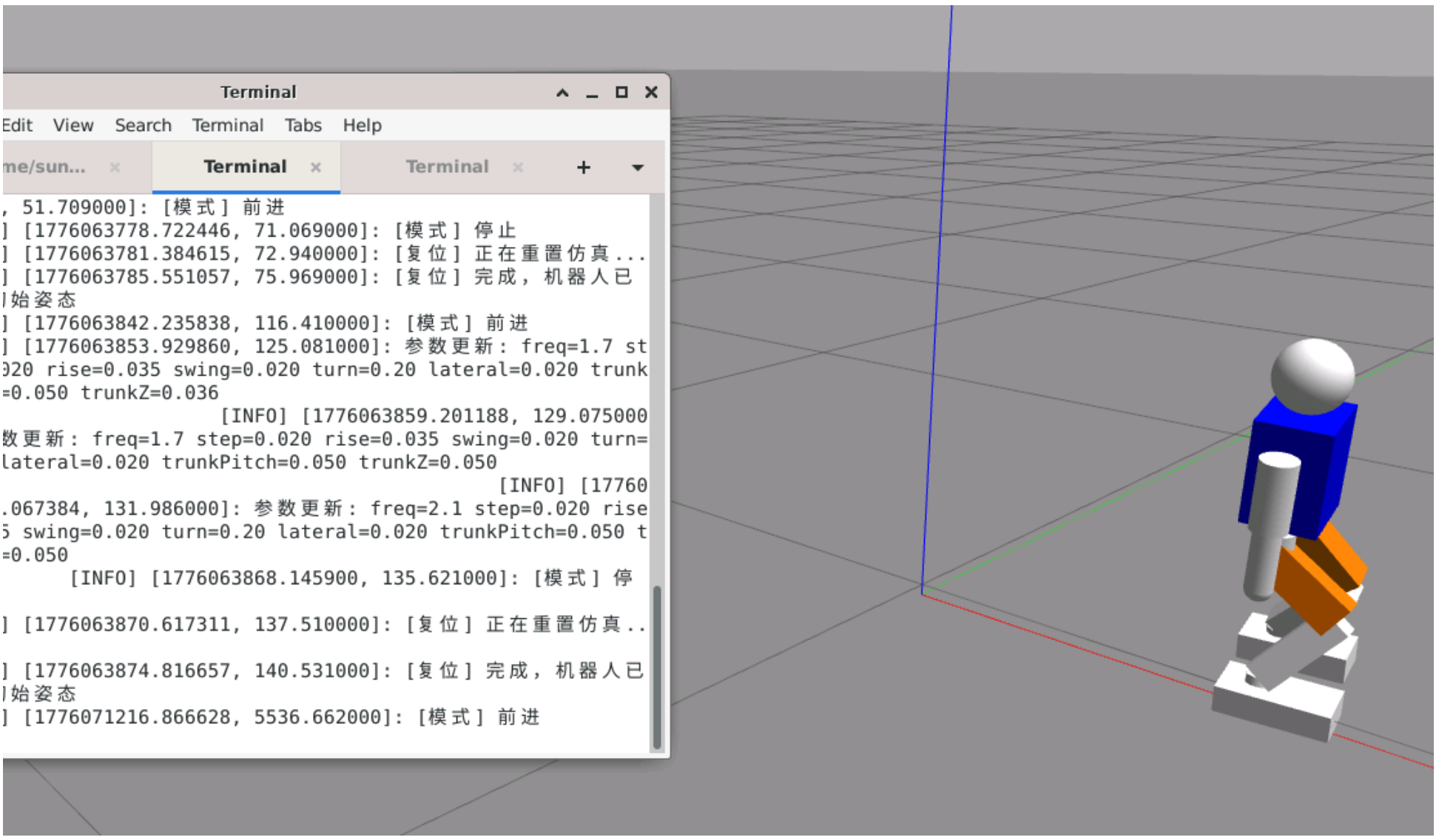
1.1 步态参数实验与记录

1. 启动仿真环境和步态节点，使用默认参数让机器人前进行走，确认步态基本稳定；
2. 自制表格逐个调整步态参数，每个参数至少测试 3 个不同值（含默认值），记录机器人的行走表现（稳定 / 轻微晃动 / 摔倒）：

参数	测试值 1	表现	测试值 2	表现	测试值 3	表现
`freq`						
`step_gain`						
`rise_gain`						
`swing_gain`						
`trunk_pitch`						
`trunk_z_offset`						

3. 找到你能让机器人稳定行走的最大步幅，记录此时的完整参数组合：
   1. `freq = ____, step_gain = ____, rise_gain = ____, swing_gain = ____, trunk_pitch = ____, trunk_z_offset = ____`
4. 分析：你觉得哪个参数对稳定性影响最大？为什么？

1. *采用默认参数，机器人行走正常，如下图所示：*



2. *调整不同步态参数，得到机器人的行走表现如下表（其中测试值 1 位默认参数。测试值 2/3 为针对每个变量单独调整，其他变量保持不变的时候的测试效果）：*

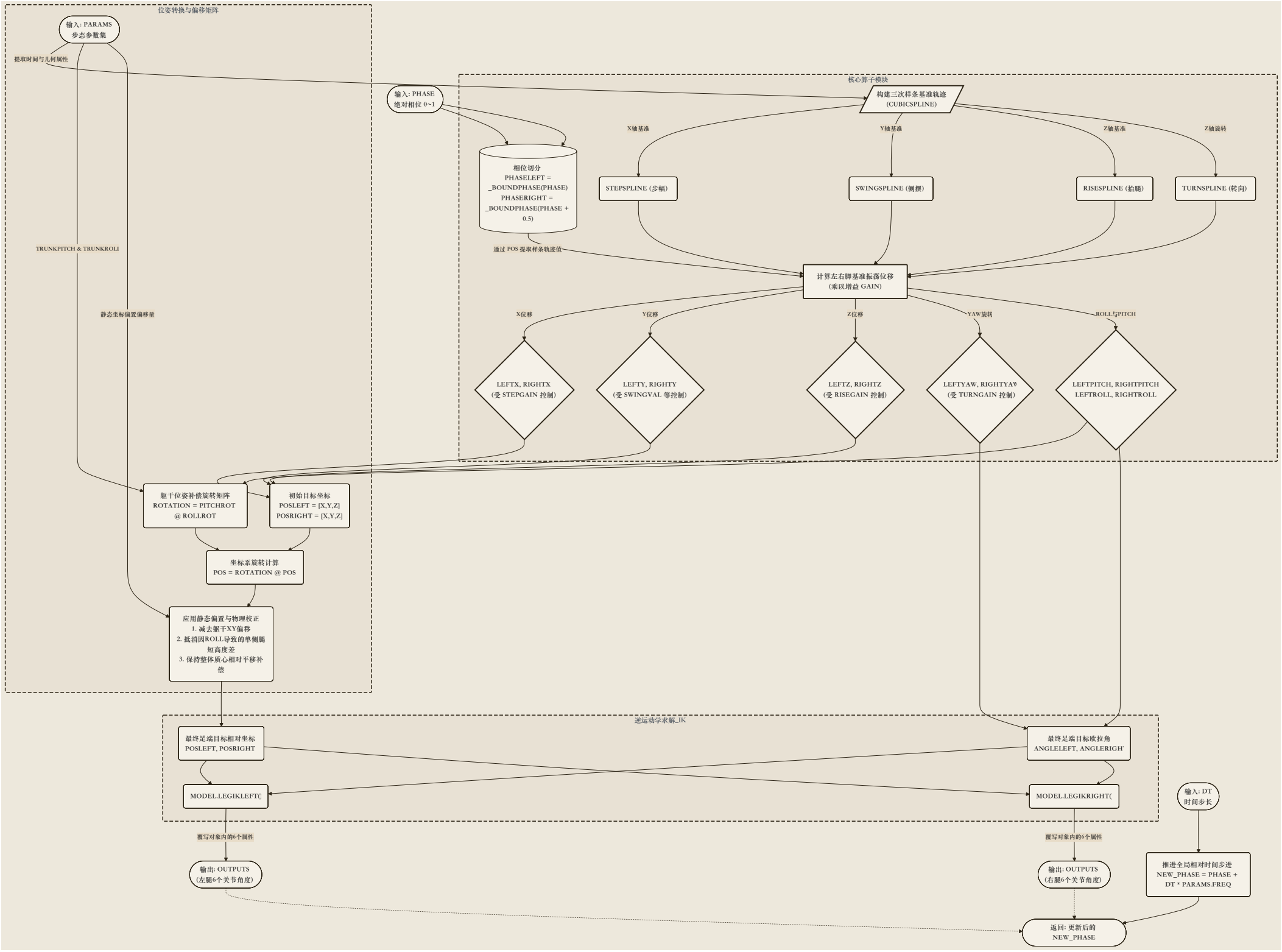
参数	测试值1(base)	表现	测试值2	表现	测试值3	表现
freq	2.078	稳定	3	稳定，晃动幅度一般	0.5	稳定，但晃动较大
step_gain	0.02		0	稳定	0.04	稳定
rise_gain	0.035		0.01	稳定，转动速度慢	0.06	稳定
swing_gain	0.02		0	摔倒	0.05	摔倒
turn_gain	0.2		0	稳定，但转不了方向	0.3	摔倒
lateral_gain	0.02		0	稳定	0.03	摔倒
trunk_pitch	0.05		-1	身体后倾，将要摔倒	0.3	身体前倾，稳定
trunk_z_offset	0.02		0	无法求解	0.03	摔倒

3. *在保证机器人稳定行走最大步幅的前提下，得到的参数为：* `freq = 0.5, step_gain =0.04, rise_gain =0.06, swing_gain =0.02 , trunk_pitch = 0.05, trunk_z_offset =0.02`
4. `trunk_pitch` *(躯干前倾角) 和* `swing_gain` *(侧摆幅度) 对机器人稳定性影响较大。*
   - *trank_pitch: 机器人在迈步时会有向前的惯性，如果上半身不保持一个合适的俯仰角（Pitch），机器人在抬腿的一瞬间极易因为重心靠后而向后仰倒；或者因为前倾太大而向前扑倒。*
   - *swing_gain: 双足机器人在单腿支撑期（即另一条腿悬空时），为了不向悬空侧跌倒，必须将躯干中心（重心）横向平移到支撑脚的正上方。swing_gain 控制的就是躯干左右晃动的幅度。*

1.2 步态引擎代码结构分析

> 阅读 `ikwalk_engine.py`，回答以下问题：
>
> 1. 画出 `walk()` 函数的数据流图。从输入（params, dt, phase）到输出（outputs 中的 12 个关节角），标出每一步的关键变量名和计算方式；
> 2. 四条样条各自的 `addPoint` 关键点是什么含义？以 `stepSpline` 为例，解释四个 `addPoint` 调用中相位、值、速度三个参数分别控制什么；
> 3. `hip_pitch` 和 `ankle_pitch` 为什么要取负号？如果不取负号，机器人行走时会出现什么问题？

1.2.1 walk() 函数的数据流图如下所示



解释如下：

第 1 步：相位切分（Phase splitting）

步态控制使用归一化相位 `(0.0 ~ 1.0)` 来表示一个完整的行走周期。
计算：
- 左腿相位： `phaseLeft = _boundPhase(phase)`
- 右腿相位：因为双腿交替，所以右腿落后/超前半个周期 `phaseRight = _boundPhase(phase + 0.5)`

第 2 步：样条曲线构造（Spline Construction）

机器人的四个自由度动作基于特定轮廓的三次样条曲线（`CubicSpline`）。使用 `addPoint(time, position, velocity)` 添加控制点。

- `stepSpline` (X 轴向前)：控制脚底在空中往前飞、在地上往后蹬。
- `swingSpline` (Y 轴左右侧摆)：保证重心跨越到支撑腿上方。
- `riseSpline` (Z 轴上下抬放)：控制空中摆动腿的抬离和落地。
- `turnSpline` (Z 轴 Yaw 旋转)：控制原地踏步时的身体转向。

第 3 步：脚部笛卡尔坐标与姿态振荡求值（Oscillation Computation）

从样条曲线中拿到值后，乘以相应的系统总开关 (`enabledGain`) 与该动作的增益限制，获得原始的动态位姿。

- X, Z 位移与 Yaw 旋转：
  - 直接取样： `leftX = params.enabledGain * params.stepGain * stepSpline.pos(phaseLeft)`
  - `rightX`，`leftZ`，`rightZ`，`leftYaw`，`rightYaw` 用各自的 Gain 与样条算式同理。
- Y（侧向）位移：稍微复杂一点，由于重心左右摇摆，需要一个单独的摇摆值：
  - `swingVal = params.enabledGain * params.swingGain * swingSpline.posMod(0.5 + phaseLeft + params.swingPhase)`
  - `leftY` 和 `rightY` 将叠加 `swingVal`，同时叠加侧滑步幅 (`lateralGain`) 和机器人固有的分腿宽度 (`footYOffset`)。
- Roll / Pitch 欧拉角：
  - Pitch (躯干俯仰) 保持恒定前倾： `leftPitch = params.trunkPitch`

- Roll (躯干横滚) 跟随 `swingSpline` 联动： `leftRoll = rollVal` 。

第 4 步：应用姿态矩阵与空间投影纠正 (Trunk Orientations Matrix & Shifts)

为了让机器人能倾斜着身体行走（比如为了防摔设定的 `trunkPitch` = 0.05），需要进行矩阵乘法，并在世界笛卡尔坐标下手控补偿一些物理误差：

- 坐标系合并： `posLeft = np.array([leftX, leftY, leftZ])`
- 计算旋转矩阵：基于 EulerY(Pitch) 和 EulerX(Roll) 构造 `rotation = pitchRot @ rollRot` 。
- 姿态偏置： `posLeft = rotation @ posLeft` （将预期的步态轨迹按躯干前倾的程度进行了投射翻转）。
- 重力高度校正：当发生明显侧倾（ `rollVal` > 0）时，外部悬空脚和支撑脚高度会产生误差，这里用了三角函数补偿 `deltaLen = model.feetDistance() * math.tan(rollVal)` 改写了 `posRight[2]` 或 `posLeft[2]` 。

第 5 步：解析逆运动学 (Analytical Inverse Kinematics)

终于生成了每个脚端相对于躯干髋部的 3D 位置 `pos[X,Y,Z]` 和目标欧拉角向 `angle[Pitch,Roll,Yaw]` 。

- 求解调用：调用内置的解析学类方法：
  - `successLeft = model.legIkLeft(posLeft, angleLeft, EulerType.EulerPitchRollYaw, outputs)`
- 算法原理 (12 步 IK 法)：在 `legIkLeft` 内部，会将目标欧拉角度转化成 `3x3` 旋转网络，连同 `Vector3D` 坐标输入给 `LegIK_IK.compute(...)` 。由于双足机器人的腿是串联的 6-DOF（通常为：髋 yaw，髋 roll，髋 pitch，膝关节，踝 pitch，踝 roll），算法分别通过几何代数和余弦定理等直接写死 12 个计算步骤（代码中的 `Step 1` 到 `Step 12` ），算出 6 个旋转轴所需的弧度值。
- 写入输出：求出的结果自动打包赋予并更新到 `outputs.left_hip_yaw` 乃至 `outputs.right_ankle_roll` 共 12 个参数上。

第 6 步：更新全系统相位

- 时间推演：如果 IK 计算成功没有出错返回 false，则累加时间： `phase += dt * params.freq` 并且取模归化到 `0~1` 内并返回。供下一次 `walk()` 使用

1.2.2 `addPoint` 关键点含义

在 `ikwalk_engine.py` 中，步态引擎本质上是通过定义三次样条（Cubic Spline）来规划各个自由度在一个周期（0.0 ~ 1.0）内的运动规律。
`addPoint(time, position, velocity)` 函数用于在样条曲线上打“关键帧”，这三个参数的作用如下：

1. `time` (时间/相位)：表示位于周期内的什么时刻（从 0.0 到 1.0）。决定了动作什么时候发生。
2. `position` (规范化位置值)：表示在这个时刻的归一化幅度（通常在 -1.0 到 1.0 之间）。稍后会乘上 `Gain` （如设定的步幅 0.02m），决定了动作幅度有多大。
3. `velocity` (速度/切线斜率)：表示在这个时刻曲线的导数。它控制曲线在该点的“切线方向”，决定了动作的爆发力或平滑度（是匀速穿过，还是急起急停）。

以 `stepSpline` 为例深度解析

`stepSpline` 规划的是机器人脚底 X 轴（前进/后退）的运动。它的周期可以分为两段：支撑期（脚踩在地上向后滑，推动身体向前）和摆动期（脚离地向前迈出）。
它的 4 个 `addPoint` 完美定义了这个物理过程：

```
# stepLength 通常是 0.5（代表周期有一半时间脚在地上踩着）
stepSpline.addPoint(0.0, 0.5, -1.0 / stepLength)
stepSpline.addPoint(stepLength, -0.5, -1.0 / stepLength)
stepSpline.addPoint(stepLength, -0.5, params.stepUpVel)
stepSpline.addPoint(1.0, 0.5, -params.stepDownVel)
```

1. 第 1 个点：支撑期开始（前脚掌刚落地）
   - `(0.0, 0.5, -1.0 / stepLength)`
   - 含义：在相位 `0.0` 时刻，脚在身体最前方（位置 `0.5` ）。速度配置为严格的负匀速（ `-1.0 / stepLength` ），即脚开始贴着地面向后平移，这在视觉上体现为躯干被脚带着向前平稳推进。
2. 第 2 个点：支撑期结束（后脚跟即将离地）
   - `(stepLength, -0.5, -1.0 / stepLength)`
   - 含义：在相位 `stepLength` （如 0.5）时刻，脚已经滑到了身体最后方（位置 `-0.5` ）。注意此处的速度依然是强制匀速。这确保了在双脚都在地上的瞬间，左右脚拉扯地面的速度是完全一致的，不会发生两腿互相撕扯的力学矛盾。
3. 第 3 个点：摆动期开始（瞬间抬腿前摆）
   - `(stepLength, -0.5, params.stepUpVel)`
   - 含义：时间点和第 2 个点一模一样！ 但速度从“向后匀速”发生阶跃，变成了巨大的正向速度（ `params.stepUpVel` ，通常>0）。这使得轨迹瞬间拐弯弯折：脚刚一离地，立刻以极大的加速度往前甩。
4. 第 4 个点：摆动期结束（跨步完成，准备压地）
   - `(1.0, 0.5, -params.stepDownVel)`
   - 含义：到时刻 `1.0` ，脚重新回到了最前方位置（ `0.5` ）。这里的负向速度（ `-params.stepDownVel` ），意味着脚在即将触地时，会有一个如同“砸向”地面或向后扣的急停减速效果，避免脚掌落地时过度滑移和扰动。

其他三条样条的“关键点”核心含义速览

- `riseSpline` (Z 轴高度——控制抬腿高度)：
  - 在 `0` 到 `stepLength` 时，值为 `0` 。这就封锁了脚底高度，强制在支撑相（踩地时）高度紧贴地面。
  - 到了 `(1.0+stepLength)/2.0` （空中最中间时刻），高度达到 `1.0` （最高点）。控制着跨步的抛物线圆润程度。
- `swingSpline` (Y 轴侧向——控制左右摇摆转移重心)：
  - 用了多达 10 个控制点。因为它的核心诉求是**“在某一条腿上悬停一小会儿”**。
  - 其中加入了 `swingPause` 参数，在最高点或最低点强制保持水平直线（位置保持不变，速度为 `0` ），让支撑腿可以稳住身体让悬空腿有足够时间往前摆。
- `turnSpline` (Yaw 轴转动——控制原地打转)：
  - 它的关键点逻辑是：只要脚在地上（0 到 stepLength-0.5 期间），转动的值就被强制置为 `0` 。因为开环控制绝对不能让原本踩在实地上的脚掌发生自转（会跟地面发生极强的摩擦碰撞），转动动作必须全部在空中摆动相完成。

1.2.3 `hip_pitch` 和 `ankle_pitch` 为什么取负号

在双足机器人的逆运动学（IK）控制中，`hip_pitch`（髋部俯仰）和 `ankle_pitch`（踝部俯仰）需要在赋值给最终电机模型前取负号，这是为了将“纯粹的数学几何解”映射到“具体的 URDF 机器人物理关节坐标系”上。
如果不取负号，机器人将会出现极度违背常理的姿态并瞬间跌倒。为了具象化，我们可以从机器人**“屈膝下蹲”这个最基本的姿态来理解：

1. 理性状态下下蹲的几何联动

当机器人为了稳定重心而微蹲（`trunk_z_offset > 0`）时，腿部的 Z 轴高度压缩，由于大腿和小腿长度固定（物理连杆无法压缩），所以必须弯曲。 为了让躯干保持直立，且脚底绝对贴合地面，正确的俯仰角应当呈现**“Z”字折叠**：

1. 膝关节 (Knee)：必须向后弯。
2. 髋关节 (Hip Pitch)：必须让大腿向前转动（相对于骨盆），这样才能补偿膝盖后弯，不让身体后仰。
3. 踝关节 (Ankle Pitch)：必须让小腿向前倾斜（相对于脚面），这样才能脚掌贴地，而不是踮起脚尖。

2. 数学方程域 vs 物理模型域 的坐标系冲突

- 在 IK 算法纯解析求解域中（`ikwalk_engine.py` 的 12 步解算法）： 算法在求解三角形（利用余弦定理求 `theta`）时，出于严密的笛卡尔直角平面几何推导习惯，通常设定顺时针或逆时针的一套绝对基准。算出来的 `theta[2]` (Hip) 和 `theta[4]` (Ankle) 往往是个符合数学公式的正负值。
- 在真实的机器人/仿真 URDF 文件中（依据右手法则）： 机器人在 URDF 里对每个电机的 `<axis>` 进行了严格定义。通常情况下，沿着机器人的横向 Y 轴，基于右手螺旋定则，大腿向后摆、脚尖向下压** 被定义为正方向（不同机器人厂商的规范可能不同，Sigmaban/当前项目的 URDF 设置正好如此）。

3. 如果不取负号，行走时会出现什么灾难性问题?

如果你在 `_setIKResult` 函数中去掉了那个负号（例如写成 `outputs.left_hip_pitch = result.theta[2]`），当机器人试图迈步并微蹲时：

- 躯干会向后折叠（下腰）：本该向前旋转弥补膝盖弯曲的髋部，现在向后旋转，躯干直接向后方瘫倒。
- 脚掌会直接“扎进”地面（芭蕾舞踮脚）：本该保持水平的脚底，由于方向反转，脚尖会狠狠向下戳地面。在 Gazebo 仿真里，这种刚体的穿模操作会触发巨大的反作用力爆炸（物理引擎解算碰撞发散），机器人会宛如“踩雷”一般直接被弹飞到天上或者疯狂抽搐。

总结

取负号的作用，就是一个标准的“软硬连接适配层”。它抹平了“抽象算法中的角度定义”和“具体 3D 模型电机正反转”之间的差异，保证了双脚能始终平行贴着地面踩稳，躯干保持我们设定的直立状态。

基础作业提交要求

> 将以上内容整理为 **一份 PDF**，包含：
>
> 1. 参数实验表格（填写完整）
> 2. 最大步幅参数组合
> 3. 代码结构分析（数据流图 + 思考题回答）
> 4. Gazebo 机器人行走截图（至少 2 张：默认参数稳定行走 + 摔倒状态各 1 张）